UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación · Carrera de Ingeniería de Software

Prueba de diagnóstico de conocimientos previos – Clave de respuestas

Aplicaciones Distribuidas (ISR-701) · Séptimo nivel · Período 2026–2027

Código:	EV-DIAG-01 (diagnóstica, no calificada)	**Duración:**	40 minutos
Docente:	Ing. Gleiston C. Guerrero Ulloa, PhD	**Fecha:**	_____
Estudiante:	_____	**Paralelo:**	_____

Instrucciones

- Esta prueba explora los conocimientos de asignaturas anteriores que necesitará en Aplicaciones Distribuidas; su resultado orienta la nivelación y no afecta su calificación.
- Las preguntas de **selección única** tienen cuatro alternativas y solo una es correcta.
- Las preguntas de **selección múltiple** tienen cinco alternativas; más de una es correcta. Marque todas las que correspondan y ninguna incorrecta.
- Trabaje individualmente, sin apuntes, sin dispositivos y sin herramientas de inteligencia artificial. Si no sabe una respuesta, déjela en blanco: un diagnóstico honesto le beneficia.

Bloque A. Sistemas operativos y concurrencia

1. [Selección única: marque una sola alternativa]

Un programa en ejecución crea varias unidades de ejecución que comparten el mismo espacio de direcciones, el mismo montículo (heap) y los mismos archivos abiertos, aunque cada una conserva su propia pila y su propio contador de programa. ¿Cómo se denominan esas unidades?

☐ a) Procesos hijos creados mediante la llamada `fork()`.

☐ b) Segmentos de memoria compartida (*shared memory*).

☐ c) Hilos (*threads*). ✓ **Correcta**

☐ d) Demonios del sistema (*daemons*).

Justificación: Los hilos de un mismo proceso comparten código, datos, montículo y descriptores, pero cada uno tiene pila, registros y contador de programa propios. Un proceso creado con `fork()` recibe una copia separada del espacio de direcciones; un segmento de memoria compartida es una región, no una unidad de ejecución; un demonio es un proceso en segundo plano.

2. [Selección única: marque una sola alternativa]

Dos hilos ejecutan simultáneamente el método `tarea()` sobre el mismo objeto. Al terminar ambos, el valor impreso suele ser menor que 200000 y cambia entre ejecuciones. ¿Cuál es la causa?

```
class Contador {
    private int valor = 0;
    void tarea() { for (int i = 0; i < 100000; i++) { valor++; } }
}
```

☐ a) El recolector de basura pausa los hilos y descarta los incrementos que estaban pendientes en ese momento.

☐ b) La instrucción `valor++` consta de leer, sumar y escribir; los pasos de ambos hilos se intercalan y algunas escrituras sobrescriben a otras. ✓ **Correcta**

☐ c) Cada hilo trabaja sobre una copia privada del objeto y, al finalizar, la JVM conserva solo la copia del último hilo.

☐ d) El tipo `int` se desborda al acercarse a 200000 y el valor vuelve a empezar desde un número menor.

Justificación: Es una condición de carrera: `valor++` no es atómica, por lo que dos hilos pueden leer el mismo valor y escribir el mismo resultado, perdiendo una actualización [1]. El recolector no altera variables vivas, la JVM no duplica objetos por hilo y el máximo de `int` supera los dos mil millones.

3. [Selección múltiple: marque todas las alternativas que correspondan]

Un bloque de código modifica una lista compartida y se requiere que, en cualquier instante, solo un hilo pueda estar ejecutándolo. ¿Qué recursos de Java garantizan ese comportamiento si se usan correctamente?

☐ a) Declarar el bloque como `synchronized` sobre un mismo objeto monitor. ✓ **Correcta**

☐ b) Declarar la lista con el modificador `volatile`.

☐ c) Rodear el bloque con `lock()` y `unlock()` de un `ReentrantLock` compartido. ✓ **Correcta**

☐ d) Asignar al hilo que modifica la lista la prioridad `Thread.MAX_PRIORITY`.

☐ e) Rodear el bloque con `acquire()` y `release()` de un `Semaphore` creado con un único permiso. ✓ **Correcta**

Justificación: `synchronized`, `ReentrantLock` y un semáforo binario impiden que dos hilos entren a la vez a la sección crítica [1]. `volatile` solo garantiza visibilidad de la referencia, no exclusión; la prioridad es una sugerencia al planificador y no impide la ejecución simultánea.

4. [Selección única: marque una sola alternativa]

El hilo H1 retiene el recurso A y espera el recurso B; al mismo tiempo, el hilo H2 retiene B y espera A, y ambos quedan detenidos indefinidamente. ¿Qué decisión de diseño impide que esta situación vuelva a producirse sin eliminar el uso de ambos recursos?

☐ a) Establecer un orden global y obligar a que todos los hilos soliciten los recursos siempre en ese mismo orden. ✓ **Correcta**

☐ b) Elevar la prioridad de H1 para que el planificador lo ejecute antes que a H2.

☐ c) Declarar los recursos A y B como `volatile` para que ambos hilos vean su estado actualizado.

☐ d) Aumentar el número de hilos del grupo (*thread pool*) para que haya más hilos disponibles.

Justificación: Es un interbloqueo (*deadlock*); requiere simultáneamente exclusión mutua, retención y espera, no expropiación y espera circular [2]. Ordenar globalmente la adquisición rompe la espera circular. La prioridad, la visibilidad o más hilos no rompen ninguna de las cuatro condiciones.

Bloque B. Redes y comunicación

5. [Selección múltiple: marque todas las alternativas que correspondan]

¿Qué características corresponden al protocolo de la capa de transporte sobre el que funcionan por defecto HTTP/1.1 y las conexiones JDBC hacia un servidor de base de datos?

☐ a) Conserva los límites de cada mensaje: lo que el emisor envía en una escritura llega en una sola lectura del receptor.

☐ b) Antes de intercambiar datos, ambos extremos realizan un intercambio inicial de tres segmentos para sincronizar números de secuencia. ✓ **Correcta**

☐ c) Determina la ruta de los paquetes entre redes distintas a partir de las direcciones IP de origen y destino.

☐ d) Entrega los bytes a la aplicación en el mismo orden en que fueron enviados y reenvía los que se pierden. ✓ **Correcta**

☐ e) Prescinde de establecer una conexión previa, lo que reduce la latencia del primer envío.

Justificación: El protocolo es TCP: orientado a conexión (saludo de tres vías), con entrega ordenada y retransmisión [3]. TCP es un flujo de bytes y no conserva límites de mensajes (eso es propio de UDP); el enrutamiento por IP corresponde a la capa de red; operar sin conexión es característica de UDP.

6. [Selección única: marque una sola alternativa]

Un servidor escucha en el puerto 8080 y mantiene abiertas al mismo tiempo 50 conexiones con clientes distintos. ¿Qué información utiliza el sistema operativo del servidor para saber a cuál de esas conexiones pertenece cada segmento recibido?

☐ a) La dirección física (MAC) de la tarjeta de red de cada cliente.

☐ b) Un puerto distinto que el servidor asigna a cada cliente, a partir del 8081, al aceptar la conexión.

☐ c) La cookie de sesión que el cliente incluye en cada solicitud.

☐ d) La combinación de dirección IP y puerto del cliente junto con la dirección IP y puerto del servidor. ✓ **Correcta**

Justificación: Cada conexión TCP se identifica por el par de *sockets* de ambos extremos (IP y puerto locales y remotos) [3]; por eso todas comparten el puerto 8080 del servidor. La MAC no sobrevive al primer enrutador, el servidor no cambia de puerto por cliente y la cookie pertenece a la capa de aplicación, no al sistema operativo.

7. [Selección única: marque una sola alternativa]

Un componente recibe todas las peticiones de los clientes y decide a qué servidor reenviarlas según la ruta solicitada, por ejemplo `/api/pedidos` frente a `/api/clientes`. ¿En qué capa del modelo TCP/IP debe operar ese componente para tomar esa decisión?

☐ a) Capa de transporte.

☐ b) Capa de aplicación. ✓ **Correcta**

☐ c) Capa de internet (red).

☐ d) Capa de enlace de datos.

Justificación: La ruta de la URL forma parte del mensaje HTTP, que solo es legible en la capa de aplicación (equilibradores de capa 7 o pasarelas de API). En la capa de transporte solo se ven puertos e IP; en las capas inferiores, direcciones de red o físicas.

Bloque C. Servicios web y desarrollo de aplicaciones

8. [Selección múltiple: marque todas las alternativas que correspondan]

Si una misma solicitud se envía varias veces por un reintento automático, ¿con qué métodos HTTP el efecto final en el servidor es el mismo que si se hubiera enviado una sola vez, según la especificación de la semántica de HTTP?

☐ a) `POST`

☐ b) `GET` ✓ **Correcta**

☐ c) `PUT` ✓ **Correcta**

☐ d) `PATCH`

☐ e) `DELETE` ✓ **Correcta**

Justificación: RFC 9110 define como idempotentes a `GET`, `HEAD`, `OPTIONS`, `TRACE`, `PUT` y `DELETE` [4]. `POST` puede crear un recurso nuevo en cada envío y `PATCH` no está definido como idempotente.

9. [Selección única: marque una sola alternativa]

Un cliente solicita un recurso protegido de una API sin incluir ningún token ni credencial en la petición. ¿Qué código de estado debe devolver el servidor?

☐ a) `403`

☐ b) `404`

☐ c) `401` ✓ **Correcta**

☐ d) `400`

Justificación: El código 401 indica que faltan credenciales válidas y que el cliente debe autenticarse; el 403 se reserva para un cliente ya identificado que no tiene permiso [4]. El 404 indica que el recurso no existe y el 400 que la petición está mal formada.

10. [Selección múltiple: marque todas las alternativas que correspondan]

¿Qué propiedades forman parte de las restricciones del estilo arquitectónico REST, tal como fue definido por su autor?

☐ a) Cada petición contiene toda la información necesaria para ser atendida, sin depender de datos de sesión guardados en el servidor. ✓ **Correcta**

☐ b) Las representaciones de los recursos deben intercambiarse en formato JSON.

☐ c) Las respuestas indican explícitamente si pueden almacenarse temporalmente para reutilizarse. ✓ **Correcta**

☐ d) Toda comunicación debe cifrarse mediante HTTPS.

☐ e) Los recursos se identifican mediante URI y se manipulan a través de un conjunto común y uniforme de operaciones. ✓ **Correcta**

Justificación: Fielding define REST mediante cliente-servidor, ausencia de estado de sesión en el servidor, caché, interfaz uniforme, sistema en capas y código bajo demanda [5]. REST no impone JSON (admite cualquier representación) ni HTTPS, aunque este último sea una buena práctica de seguridad.

11. [Selección única: marque una sola alternativa]

Un controlador Spring Boot devuelve directamente una entidad JPA `Cliente` que tiene una lista de `Pedido`, y cada `Pedido` referencia de vuelta a su `Cliente`. Al consumir el servicio desde una aplicación móvil se obtiene un `StackOverflowError` durante la conversión a JSON. ¿Qué solución es la adecuada?

☐ a) Devolver objetos de transferencia (DTO) sin la referencia de retorno, o excluir uno de los lados de la relación de la conversión a JSON. ✓ **Correcta**

☐ b) Aumentar el tamaño de la pila de la JVM con el parámetro `-Xss` para que la conversión logre completarse.

☐ c) Cambiar la estrategia de carga de la relación a `FetchType.EAGER` para que los pedidos se obtengan junto con el cliente.

☐ d) Hacer que ambas entidades implementen la interfaz `Serializable`.

Justificación: La relación bidireccional provoca una recursión infinita (cliente, pedidos, cliente…) al convertir a JSON. Se corta usando DTO o anotaciones como `@JsonIgnore` o `@JsonManagedReference`/`@JsonBackReference`. Una pila mayor solo retrasa el error, la carga ansiosa no elimina el ciclo y `Serializable` se refiere a la serialización binaria de Java, no a JSON.

Bloque D. Bases de datos

12. [Selección múltiple: marque todas las alternativas que correspondan]

Una transferencia bancaria descuenta 100 dólares de la cuenta X y los acredita en la cuenta Y dentro de una misma transacción de un gestor relacional. ¿Qué afirmaciones describen correctamente las garantías ACID?

☐ a) Si el crédito en Y falla, el débito ya aplicado en X se deshace y ninguna de las dos operaciones queda registrada. ✓ **Correcta**

☐ b) La consistencia garantiza que todas las réplicas de la base de datos muestren el mismo saldo en el mismo instante.

☐ c) Una vez confirmada la transacción, sus cambios permanecen aunque el servidor se apague inmediatamente después. ✓ **Correcta**

☐ d) El aislamiento obliga a que el gestor ejecute las transacciones de una en una, sin concurrencia.

☐ e) Otra transacción que se ejecute en paralelo no observa el estado intermedio en que el dinero salió de X pero aún no llegó a Y. ✓ **Correcta**

Justificación: Las alternativas a), c) y e) describen atomicidad, durabilidad y aislamiento. La C de ACID se refiere al respeto de las reglas de integridad de la base, no a la igualdad entre réplicas (esa idea corresponde a la C del teorema CAP, que se estudiará en el curso). El aislamiento se logra mediante bloqueos o versiones y permite concurrencia; la ejecución en serie es solo una forma de razonarlo [6].

13. [Selección única: marque una sola alternativa]

La transacción T1 consulta un saldo y obtiene 100. Mientras T1 sigue abierta, la transacción T2 cambia ese saldo a 50 y confirma. Luego T1 vuelve a consultar la misma fila y obtiene 50. ¿Qué fenómeno ocurrió y cuál es el nivel de aislamiento mínimo del estándar SQL que lo evita?

☐ a) Lectura sucia; se evita con `READ COMMITTED`.

☐ b) Lectura fantasma; se evita con `SERIALIZABLE`.

☐ c) Lectura no repetible; se evita con `REPEATABLE READ`. ✓ **Correcta**

☐ d) Actualización perdida; se evita con `READ UNCOMMITTED`.

Justificación: T1 leyó dos veces la misma fila con resultados diferentes debido a un cambio confirmado por otra transacción: lectura no repetible, que `REPEATABLE READ` impide [6]. No es lectura sucia porque T2 había confirmado; no es fantasma porque no aparecen filas nuevas; `READ UNCOMMITTED` no evita ninguna anomalía.

14. [Selección única: marque una sola alternativa]

La tabla `estudiante` tiene un índice de árbol B sobre la columna `apellido`; sin embargo, la consulta `SELECT * FROM estudiante WHERE apellido LIKE '%ez';` recorre la tabla completa. ¿Por qué?

☐ a) El operador `LIKE` nunca puede aprovechar un índice, sin importar el patrón utilizado.

☐ b) El índice solo se usaría si se hubiera declarado como `UNIQUE`.

☐ c) Las columnas de tipo `VARCHAR` no admiten índices de árbol B.

☐ d) Al desconocerse el inicio del texto buscado, el gestor no puede usar el orden del índice para delimitar el rango de búsqueda. ✓ **Correcta**

Justificación: El árbol B ordena los valores por su prefijo; un patrón como `'Gu%'` sí puede usar el índice, pero un comodín al inicio impide localizar un rango y obliga a revisar todas las filas. Los índices no necesitan ser únicos y los `VARCHAR` sí son indexables.

Bloque E. Ingeniería de software, seguridad y herramientas

15. [Selección múltiple: marque todas las alternativas que correspondan]

Dos integrantes del equipo modificaron las mismas líneas de `PedidoService.java` en ramas distintas y ahora intentan integrar su trabajo. ¿Qué afirmaciones son correctas?

☐ a) `git pull -force` integra automáticamente ambas versiones conservando los cambios de los dos integrantes.

☐ b) `git merge` se detiene y marca en el archivo las zonas en disputa con los delimitadores `<<<<<<<`, `=======` y `>>>>>>>`. ✓ **Correcta**

☐ c) `git rebase` reescribe los commits de la rama que se reaplica, por lo que estos obtienen identificadores (hash) nuevos. ✓ **Correcta**

☐ d) `git stash` incorpora los cambios del compañero en la copia de trabajo local.

☐ e) Tras editar el archivo, se marca como resuelto con `git add` y se concluye con `git commit` o con `git rebase -continue`, según la operación en curso. ✓ **Correcta**

Justificación: `merge` se detiene ante el conflicto y lo marca; `rebase` crea commits nuevos con hash distinto; la resolución se registra con `git add` y se concluye con `commit` o `rebase -continue`. `git pull` no resuelve conflictos automáticamente y `git stash` guarda temporalmente los cambios propios, no los del compañero.

16. [Selección única: marque una sola alternativa]

La clase `ServicioPedido` crea dentro de su constructor, con el operador `new`, una instancia de `PedidoRepositorioMySQL`. Se desea probarla sin una base de datos real y, más adelante, cambiar a PostgreSQL sin modificar `ServicioPedido`. ¿Qué cambio de diseño lo permite?

☐ a) Declarar como `static` los métodos de `PedidoRepositorioMySQL` para invocarlos sin crear objetos.

☐ b) Hacer que `ServicioPedido` dependa de una interfaz `PedidoRepositorio` y reciba la implementación concreta como parámetro del constructor. ✓ **Correcta**

☐ c) Hacer que `ServicioPedido` herede de `PedidoRepositorioMySQL` y sobrescriba los métodos que se necesiten.

☐ d) Rodear las llamadas al repositorio con un bloque `try-catch` que devuelva datos simulados cuando falle la conexión.

Justificación: Depender de una abstracción y recibir la implementación desde fuera (inversión de dependencias e inyección por constructor) permite pasar un repositorio simulado en las pruebas o uno de PostgreSQL en producción sin tocar la clase. Los métodos estáticos y la herencia aumentan el acoplamiento con MySQL; el `try-catch` oculta fallos en lugar de resolver el diseño.

17. [Selección múltiple: marque todas las alternativas que correspondan]

Una aplicación debe guardar en su base de datos la información necesaria para verificar las contraseñas de los usuarios durante el inicio de sesión. ¿Qué prácticas son adecuadas?

☐ a) Cifrar cada contraseña con AES y conservar la clave en el servidor para poder recuperarla si el usuario la olvida.

☐ b) Aplicar una función de derivación deliberadamente lenta y configurable, como bcrypt o Argon2. ✓ **Correcta**

☐ c) Calcular un SHA-256 de una sola pasada, suficiente porque el resultado no puede revertirse.

☐ d) Combinar cada contraseña con un valor aleatorio distinto por usuario antes de procesarla y guardar ese valor junto al resultado. ✓ **Correcta**

☐ e) Codificar la contraseña en Base64 para que no sea legible a simple vista en la tabla.

Justificación: Las funciones adaptativas como bcrypt encarecen los ataques de fuerza bruta y la sal aleatoria por usuario impide el uso de tablas precalculadas y revela qué usuarios comparten contraseña [7]. El cifrado reversible expone todas las contraseñas si se filtra la clave; SHA-256 es demasiado rápido para este fin; Base64 es una codificación, no una protección.

18. [Selección única: marque una sola alternativa]

En una aplicación móvil, una llamada de red bloqueante a una API se ejecuta en el mismo hilo que atiende la interfaz gráfica, y el servidor tarda varios segundos en responder. ¿Qué ocurre?

☐ a) La interfaz deja de responder a toques y desplazamientos hasta que la llamada termina, y el sistema puede ofrecer cerrar la aplicación por no responder. ✓ **Correcta**

☐ b) La llamada se ejecuta más rápido porque el hilo de la interfaz tiene mayor prioridad en el planificador del sistema operativo.

☐ c) El sistema operativo traslada automáticamente la llamada a un hilo secundario y la interfaz continúa respondiendo.

☐ d) La llamada se cancela automáticamente a los cinco segundos y la interfaz continúa sin mostrar ningún resultado.

Justificación: El hilo principal procesa los eventos de la interfaz; si se bloquea, la aplicación se congela (en Android se muestra el diálogo ANR, y la plataforma incluso prohíbe operaciones de red en ese hilo). Por eso las llamadas de red se ejecutan de forma asíncrona (corrutinas, *futures*, `async/await`). El sistema no traslada la tarea de hilo ni la cancela por sí mismo.

19. [Selección múltiple: marque todas las alternativas que correspondan]

¿Qué afirmaciones son correctas al comparar un contenedor Docker con una máquina virtual tradicional?

☐ a) Los contenedores que se ejecutan en un mismo anfitrión comparten el núcleo (*kernel*) del sistema operativo de ese anfitrión. ✓ **Correcta**

☐ b) Cada contenedor incluye y arranca su propio núcleo del sistema operativo, independiente del anfitrión.

☐ c) Un contenedor suele iniciar en segundos porque no necesita arrancar un sistema operativo completo. ✓ **Correcta**

☐ d) El aislamiento de un contenedor lo proporciona un hipervisor de tipo 1 que emula el hardware.

☐ e) Una imagen construida para Linux en procesadores x86 se ejecuta sin cambios en cualquier arquitectura, incluida ARM.

Justificación: El contenedor es un proceso aislado mediante mecanismos del núcleo (espacios de nombres y grupos de control) y por eso arranca rápido. Las máquinas virtuales, no los contenedores, usan hipervisor y núcleo propio. Las imágenes dependen de la arquitectura del procesador; ejecutarlas en otra exige imágenes multiarquitectura o emulación.

20. [Selección única: marque una sola alternativa]

Para cada uno de los n correos de un lote nuevo debe verificarse si ya figura entre n correos registrados previamente. ¿Qué colección de Java permite que el proceso completo tenga un costo promedio de orden $O(n)$?

☐ a) Un `ArrayList` con los correos registrados, consultado con `contains()`.

☐ b) Un `TreeSet` con los correos registrados, consultado con `contains()`.

☐ c) Una `LinkedList` ordenada con los correos registrados, consultada mediante búsqueda binaria.

☐ d) Un `HashSet` con los correos registrados, consultado con `contains()`. ✓ **Correcta**

Justificación: `HashSet` ofrece consulta en tiempo constante promedio, por lo que n consultas cuestan $O(n)$. `ArrayList` produce $O(n^2)$; `TreeSet`, $O(n \log n)$; y en una lista enlazada la búsqueda binaria no es eficiente porque llegar al elemento central exige recorrer la lista.

Resumen de la clave

N.º	Tipo	Respuesta	N.º	Tipo	Respuesta
1	Única	c	11	Única	a
2	Única	b	12	Múltiple	a, c, e
3	Múltiple	a, c, e	13	Única	c
4	Única	a	14	Única	d
5	Múltiple	b, d	15	Múltiple	b, c, e
6	Única	d	16	Única	b
7	Única	b	17	Múltiple	b, d
8	Múltiple	b, c, e	18	Única	a
9	Única	c	19	Múltiple	a, c
10	Múltiple	a, c, e	20	Única	d

Criterio sugerido de puntuación diagnóstica

Cada pregunta vale 5 puntos (total 100). En selección única: 5 si es correcta, 0 en otro caso. En selección múltiple: se suman $5/k$ puntos por cada alternativa correcta marcada (k = número de correctas) y se restan $5/k$ por cada incorrecta marcada, con mínimo 0 por pregunta. Lectura por bloques: A (preguntas 1–4) concurrencia; B (5–7) redes; C (8–11) servicios web; D (12–14) bases de datos; E (15–20) diseño, seguridad y herramientas. Un bloque con menos del 60 % indica el tema que debe nivelarse en la Semana 1.

Referencias

[1] B. Goetz, T. Peierls, J. Bloch, J. Bowbeer, D. Holmes y D. Lea, *Java Concurrency in Practice*. Upper Saddle River, NJ: Addison-Wesley, 2006, ISBN: 978-0-321-34960-6.

[2] E. G. Coffman, M. Elphick y A. Shoshani, «System Deadlocks,» *ACM Computing Surveys*, vol. 3, n.º 2, págs. 67-78, 1971. DOI: 10.1145/356586.356588.

[3] W. M. Eddy, «Transmission Control Protocol (TCP),» RFC Editor, Request for Comments 9293, ago. de 2022. DOI: 10.17487/RFC9293.

[4] R. T. Fielding, M. Nottingham y J. Reschke, «HTTP Semantics,» RFC Editor, Request for Comments 9110, jun. de 2022. DOI: 10.17487/RFC9110.

[5] R. T. Fielding, «Architectural Styles and the Design of Network-based Software Architectures,» Tesis doct., University of California, Irvine, 2000.

[6] H. Berenson, P. A. Bernstein, J. Gray, J. Melton, E. J. O'Neil y P. E. O'Neil, «A Critique of ANSI SQL Isolation Levels,» en *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*, New York, NY: ACM, 1995, págs. 1-10. DOI: 10.1145/223784.223785.

[7] N. Provos y D. Mazières, «A Future-Adaptable Password Scheme,» en *Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference*, Monterey, CA: USENIX Association, 1999. dirección: https://www.usenix.org/conference/1999-usenix-annual-technical-conference/presentation/future-adaptable-password-scheme.